kairo

Product proposal

Your spreadsheet, rebuilt as a calm, secure AI workspace.

A request to run Kairo as the team's task system of record, starting with a two-sprint pilot.

Version	Status	Audience	Date
1.1	Proposed	Sponsors, team leads, IT / security	October 2026

Summary

Teams that coordinate work in a shared Excel workbook pay for it every week: statuses drift (“Pending”, “Under Review?”, blank), nobody owns half the rows, everyone can read every row, and there is no history of who changed what.

Kairo replaces that workbook with a secure, AI-assisted task workspace **without asking the team to abandon Excel on day one**: the existing sheet is imported through a validation preview, and a styled spreadsheet can be exported back at any time.

What it is	Web app: React SPA, FastAPI API, PostgreSQL
Who it is for	Admins (team leads, ops managers) and users (individual contributors)
Core promise	Spreadsheet in, secure AI workspace out
Hosting	Vercel (app + Python function) with managed PostgreSQL on Neon
Live demo	Landing page `/`, two demo accounts on `/login`

The problem

Pain in the shared workbook	Consequence
Everyone sees every row	Confidential or personal work is exposed to the whole team
Free-text statuses and priorities	Reports disagree; “Pending” and “To Do” are counted separately
Rows without owners or titles	Work silently falls through the cracks
No audit trail	Nobody can tell who changed a deadline or closed a task
No planning help	People decide what to do next by scrolling

The solution

Every capability below is implemented in the repository today.

Excel round-trip

Upload `.xlsx`, review a dry-run preview (legacy statuses mapped, rows missing a title held back), commit. Export back with native dropdowns on status and priority.

Isolated workspaces

Every task query passes one scope filter. Users see only their own, assigned or project tasks; probing anyone else's task ID returns **404**, not 403. Admins see the organisation.

Copilot

Turns a sentence into a structured task with a checklist, breaks work down, explains priority and plans the day, using only your own tasks. Works fully offline.

Chat assistants

A floating assistant creates, edits and closes tasks, comments and projects from plain language. Admins get their own, with organisation-wide reach. Deletes always wait for an explicit yes.

Four ways to work

Dense list with bulk actions, drag-and-drop board, calendar, and a task drawer with checklist, comments and activity log. `Ctrl/⌘ K` jumps anywhere.

Governance

User provisioning and roles, an organisation-wide audit trail, and productivity insights for admins.

Why this approach

- **Low switching cost.** The import and export loop lets the team migrate gradually and keeps stakeholders who still want a spreadsheet.
- **Security by construction.** Isolation lives in the data-access layer (`get_scoped_tasks_query`, `get_task_or_404`), not in hidden UI, and the assistants act through the same layer. It is covered by automated tests.
- **No lock-in on AI.** The Copilot needs no API key; the chat assistants use Groq and can be switched off by leaving the key empty.

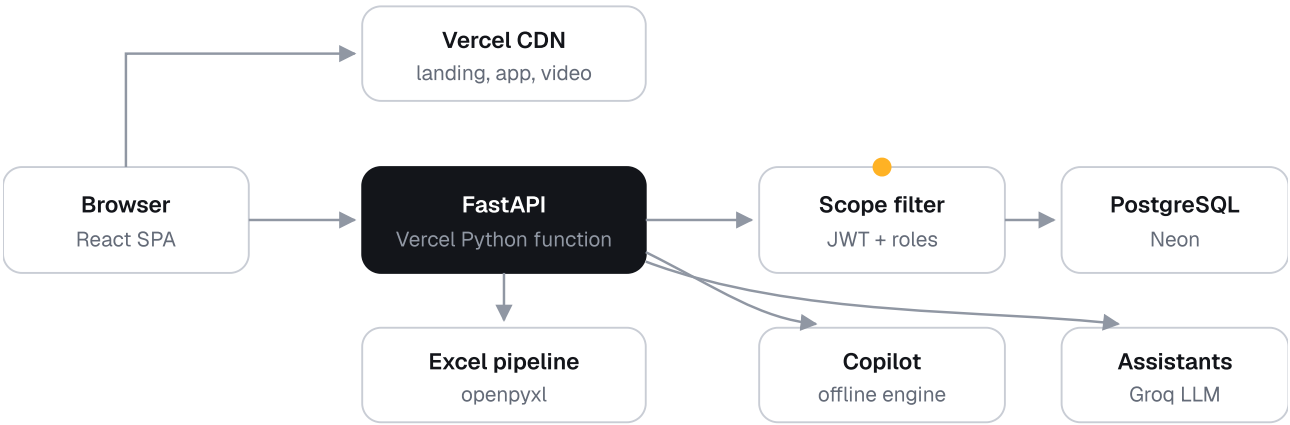
Scope and roles

- **In scope for rollout:** the capabilities above, demo data, Vercel deployment, and the documentation suite.

Out of scope (candidate follow-ups, not built): email and Slack notifications, SSO / SAML, a recurring-task UI, mobile apps, multi-organisation tenancy.

Role	Can	Cannot
Admin	See and edit every task; provision users and roles; read the audit trail and organisation insights; use the admin assistant	—
User	Create tasks; work on tasks they own, are assigned, or share through a project; use their own assistant	See other people's tasks (404); open admin pages (403)

Architecture at a glance



Every request that touches tasks, including the assistants' tool calls, passes the scope filter (saffron dot) before reaching the database.

Rollout plan

01

Deploy

Vercel project and Neon database, secrets set, demo seed loaded.

Engineering

02

Pilot

One team imports its workbook through the preview and runs on Kairo for two sprints.

Team lead

03

Review

Compare against the success measures; fix gaps.

Team lead + Engineering

04

Expand

Provision remaining users; retire the shared workbook and keep an export as archive.

Admin

Success measures

Proposed targets to agree with the sponsor. These are not measured results.

- 100% of active work items live in Kairo by the end of the pilot; the shared workbook becomes read-only.
- Zero cross-user data exposures, verified by the isolation test suite on every change.
- Every task has an owner and a valid status after import, enforced by the preview.
- Weekly stakeholder reports come from the Excel export instead of hand-edited sheets.

Risks and mitigations

Risk	Mitigation
Default secrets reach production	Startup fails on Vercel if `JWT_SECRET_KEY` is the public default
Messy legacy sheets	Dry-run preview; rows missing a title are held back and statuses normalised
AI output quality	The offline engine is predictable; drafts are always editable and never auto-committed
An assistant deletes the wrong thing	Destructive actions need an explicit “yes” from the person, enforced on the server, and every assistant turn is logged
Groq free-tier rate limits	Automatic fallback to a smaller model, then a short retry; the rest of the app is unaffected
Serverless cold starts	One lightweight function, NullPool against Neon’s pooler, static assets on the CDN
Adoption	Excel export stays available; the landing page and a 20-second tour explain the product

Cost model

Hosting runs on two managed services, each with a free tier suitable for a pilot: **Vercel** (frontend and Python function) and **Neon** (PostgreSQL). The chat assistants use **Groq**, whose free tier is rate-limited per minute. Paid tiers are needed only for production-scale traffic; check each vendor’s current pricing before the expansion step.

Decision requested

Approve the pilot (steps 1 to 3) with one team, using the deployment described in the Deployment Guide (`docs/13_DEPLOYMENT_GUIDE.md`).